# Simple Trees Beat Deep Networks at Attributing VMProtect-Protected Malware: A Leakage-Free, Forgery-Aware Evaluation

Xubo Fang*†, Haochen Dong†, Jiaqing Wang†, and Sicheng Han

***Abstract*—Static classifiers must attribute malware to threat groups even after obfuscation has rewritten its code. However, classifiers of protected malware were evaluated with splits that leak executables or builds between training and testing, and it is unknown which parts of a protected file identify its family or whether an adversary can rewrite them. Therefore, we propose a build-grouped, forgery-aware evaluation of static attribution that tests classifiers on unseen and later builds and on forged, edited, or copied headers. We compared gradient-boosted trees, eight image and byte networks, and a multi-view Transformer we developed on executables of nine advanced persistent threat groups. On unseen builds, gradient-boosted trees on the loader-relevant header fields, which VMProtect keeps, attributed protected executables most accurately, with a macro-$F_1$ of 90.9%, against 87.5% for the Transformer and at most 64.1% for the networks. The trees trained in seconds and were unaffected by forging the descriptive fields; copying another group's descriptive fields, however, lowered the macro-$F_1$ of trees on all header fields from 92.5% to 50.2%. These results show that simple gradient-boosted trees attribute VMProtect-protected malware from its loader-relevant header structure when related builds occur in training, whereas descriptive fields can be forged.**

***Index Terms*—Code obfuscation, gradient boosting, malware attribution, malware detection, portable executable, virtualization obfuscation.**

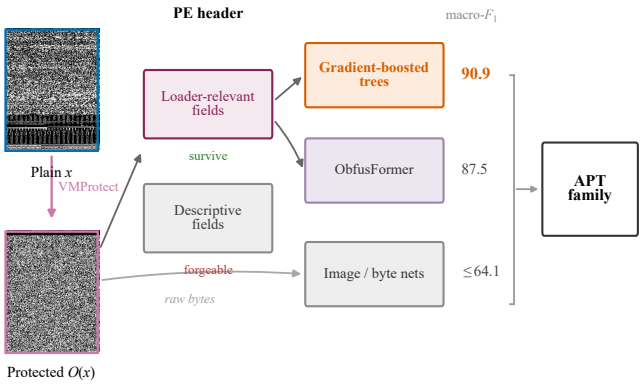



Fig. 1. Task and recommended attributor. VMProtect rewrites a plain executable $x$ into a protected version $O(x)$ (byte images). Its PE header splits into loader-relevant fields, which survive protection, and descriptive fields, which are forgeable; gradient-boosted trees on the loader-relevant fields attribute the APT family most accurately, above the multi-view Transformer (ObfusFormer) and the image and byte networks on the raw bytes (protected-test macro-$F_1$, folds grouped by build). Byte images: head windows of the median-size APT29 sample.

## I. Introduction

MACHINE learning is widely studied for malware detection and classification [1]. Static classifiers decide from the bytes of an executable without running it. They face two decisions: whether a file is malicious, and which threat group produced it. Attribution studies of advanced persistent threat (APT) malware address the second decision [2]–[4]. Commercial virtualization obfuscators such as VMProtect and Themida [5] complicate both decisions. They translate selected code into bytecode for a virtual machine embedded in the protected file [6]. In the APT executables studied here, VMProtect raised the median file size from 180 KiB to 9,648 KiB and the mean byte entropy from 4.78 to 6.93 bits per byte. Because protection rewrites the byte statistics that a static classifier learns from, accuracy on plain executables does not show whether a classifier can attribute protected ones.

Previous studies have rendered malware as images and classified the images with texture features [7], convolutional neural networks (CNNs) [8], attention modules [9], and Vision Transformers [10], [11]. For packed executables, Aghakhani *et al.* [12] showed that static classifiers did not generalize to packers absent from their training data. Han *et al.* [13] protected APT malware with VMProtect and reported an accuracy of 93.55% with Mal-CNN, a depth-wise separable CNN with spatial attention. However, three problems remain. First, in the released splits of the protected-image dataset, 820 of 1,015 test executables also contribute augmented images to the training set. Many executables of the corpus also come from one build. To our knowledge, no published study has tested attribution of protected malware on builds made after the training builds. Second, it is unknown which parts of a protected file still identify its family. An adversary can rewrite some candidate parts, such as the time stamp in the header, or copy them from malware of another group. Third, models learn from one protection state at a time, and a model trained on plain executables attributes protected executables poorly (Section V-C). Contrastive learning treats two augmented views of one image as a positive pair [14]. A defender can protect each training executable and obtain a second version of it, so this technique can be adapted to protection.

Motivated by this, we propose a build-grouped, forgery-aware evaluation of static attribution of protected malware. We first rebuild the evaluation data. We remove the unlabeled

The authors are with the Graduate School of Natural Science and Technology, Kanazawa University, Kanazawa, Japan.

†These authors contributed equally. *Corresponding author: Xubo Fang (e-mail: boukuhaku1117@gmail.com).

samples from the detection set and decode its images back into the feature values that they serialize. The attribution set pairs plain APT executables with their protected versions, its folds keep all executables of one build together, and a second split tests on the latest builds of each group, which addresses the first problem. To address the second problem, we split the portable executable (PE) header into the loader-relevant fields that Windows uses to run a program and the descriptive fields it ignores, and we test attribution on executables whose descriptive fields are forged or copied from another group and whose loader-relevant fields are edited without changing how the program runs. Gradient-boosted trees on the loader-relevant fields turn out to attribute protected executables most accurately (see Fig. 1). To test whether a learned representation of the raw file can beat them, and to address the third problem, we develop the Obfuscation-Paired Multi-View Transformer (ObfusFormer), a multi-view Transformer that reads the loader-relevant fields, the byte distribution, and two 64-KiB windows, and we train every method on each executable together with its protected version. We validate all methods on 46,245 images derived from the EMBER benchmark [15] and on 2,352 APT executables of nine groups with 2,187 protected counterparts.

Our contributions are summarized as follows:

- We formulate malware detection and family attribution of protected executables as static tasks and construct a leakage-free benchmark of 2,187 pairs of plain and VMProtect-protected executables, with folds grouped by build, a split by time, and test samples whose header fields are forged, copied from another group, or edited. Splitting augmented images at image level raised the accuracy of Mal-CNN on protected malware from 16.5% to 81.2%, so the released split rewards the recognition of individual executables.
- We show that gradient-boosted trees on the loader-relevant PE header fields attribute protected executables most accurately. On unseen builds they had a macro-$F_1$ of 90.9%, 3.4 points above the multi-view Transformer we developed and far above every baseline network, and they trained in 3 seconds per fold. VMProtect kept the time stamp, linker version, and image base of every executable, and the section table and data directories carried 81.9% of the trees' split gain. On later builds, the most accurate method reached only 61.0%.
- We develop ObfusFormer, a multi-view Transformer, as the strongest neural baseline, and a forgery-aware analysis of what an adversary can exploit. ObfusFormer reached 87.5% but did not beat the trees, and its paired loss and protection-state head changed it by less than the random seed did. Copying the descriptive fields of another group lowered the macro-$F_1$ of trees on all header fields from 92.5% to 50.2%, whereas the loader-relevant fields stayed accurate under forging.

A project page is available, with the code and the derived datasets to be released there upon publication.[1]

[1]https://panzerjagerwang.github.io/simple-trees-vmprotect/.

## II. Related Works and Existing Gaps

### A. Image and Byte Representations of Executables

Static learning-based malware analysis maps the bytes of an executable to a label without running it. Nataraj *et al.* [7] rendered malware binaries as gray-scale images and classified them with texture features, reporting 98% accuracy on 9,458 samples from 25 families. Kalash *et al.* [8] replaced the texture features with a deep CNN. Yakura *et al.* [9] introduced attention to this representation and extracted family-characteristic byte regions from the attention map. Vision Transformers for malware images added self-supervised pretraining [10] and cross-scale embedding with adaptive token masking [11]. In parallel, Raff *et al.* [16] trained a model, which they named MalConv, on the raw byte sequences of whole executables. A third line of work summarizes a whole file by its byte distribution. Saxe and Berlin [17] computed a two-dimensional histogram of byte values and local entropy and fed it, with other static features, to a deep neural network. Anderson and Roth [15] released EMBER, a benchmark of static features extracted from 1.1 million Windows executables. Its reference classifier is a gradient-boosted decision tree model [18]. A fourth line of work reads the PE header. Raff *et al.* [19] trained neural networks on the raw bytes of part of the PE header and detected malware more accurately than a model on explicitly parsed header features. Webster *et al.* [20] extracted the Rich header from 964,816 malware samples and used it to identify similar malware and builds made in different build environments.

These representations share three limitations. First, the image-based studies resize a whole file into one fixed network input, and their Transformers split that image into square patches that join bytes one image width apart in the file. Second, the byte-sequence models read a protected executable of several megabytes as one long sequence, in which the code added by the protector dominates, and the distribution features discard the byte order entirely. Third, the header studies do not separate the fields that Windows uses to load a program from descriptive fields that an adversary can rewrite. None of these studies measured which of these representations still identifies the family of an executable after virtualization.

### B. Malware Analysis Under Virtualization Obfuscation

Virtualization obfuscation translates selected program code into bytecode for a virtual machine that is generated for, and embedded in, the protected program [6]. Sharif *et al.* [21] presented Rotalume, which recovered the bytecode syntax and semantics of malware emulated with VMProtect and Code Virtualizer from x86 execution traces. Xu *et al.* [22] simplified partially virtualized binaries with VMHunt. Li *et al.* [5] extracted 1,915 verified instruction-mapping rules from four commercial obfuscators with chosen-instruction attacks. On the learning side, Aghakhani *et al.* [12] packed 50,724 executables with nine packers, none of which was VMProtect, and reported false-positive rates of up to 83.06% when tElock, PECompact, or kkrunchy was withheld from training. Gibert *et al.* [23] investigated how packing affects static machine-learning detectors, particularly visualization-based ones, and

concluded that packing limits current static systems. Han *et al.* [13] protected the executables of ten APT families with VMProtect and classified images of the protected files with Mal-CNN. For Android applications, Wu *et al.* [24] used contrastive learning to reduce the difference introduced by obfuscation. Their IFDroid classified images of function call graphs and maintained an $F_1$ of 98.4% on 69,421 obfuscated samples.

Deobfuscation recovers program semantics, but each sample requires execution traces or symbolic analysis [21], [22]. Li *et al.* [5] also found three state-of-the-art deobfuscation techniques insufficient against modern commercial obfuscators. The learning studies on Windows executables treat protected samples either as a separate dataset [13] or as a held-out condition [12]. IFDroid [24] relies on function call graphs recovered by program analysis, whereas virtualization replaces the protected functions with bytecode for an embedded interpreter [6]. None of these studies trained a byte-level classifier of Windows executables with an executable and its virtualization-protected version as a positive pair.

### C. Evaluation Protocols for Malware Learning

Evaluation protocols decide whether a reported accuracy reflects performance on unseen malware. Arp *et al.* [25] studied 30 papers from top security venues and showed how pitfalls in design and evaluation lead to unrealistic performance. Pendlebury *et al.* [26] showed that spatial and temporal bias inflated Android malware classification results. Kaufman *et al.* [27] defined leakage as the use of information about the target that should not be legitimately available during learning. Boot *et al.* [3] released 3,594 samples from 12 APT groups and compared random forests with fully connected networks on sandbox and VirusTotal reports. Gray *et al.* [4] surveyed malware authorship attribution and noted the lack of assurance about how the APT dataset of Boot *et al.* was labeled. Laurenza *et al.* [28] triaged APT malware with one-class classifiers on static features. Haddadpajouh *et al.* [29] attributed payloads of five APT families with a multi-view fuzzy consensus model and found multi-view attribution more accurate than single-view attribution.

Image datasets of APT and protected malware contain a few hundred executables per family. Han *et al.* [13] enlarged theirs with duplication and geometric augmentation. If the augmented images are divided into training and test sets image by image, images derived from one executable can fall into both sets, which is a form of the leakage defined in [27]. Detection studies on EMBER-derived data face a different pitfall: the EMBER training set contains unlabeled samples [15] that must not be merged into the benign class. On EMBER-derived data, Han *et al.* [13] also compared networks with one another rather than with the gradient-boosted reference model of the benchmark. Malware collections also contain many executables of one build, which share their header fields. A random split can place such executables in both the training and the test set. Adversaries can further modify a PE file without changing its behavior, for example by editing the DOS header or injecting content into new sections [30], [31], so header fields that a classifier relies on can be forged. To our knowledge, no published study of virtualization-protected malware has evaluated classifiers with splits grouped by source executable or by build, on builds made after the training builds, or on samples with forged, copied, or edited header fields.

## III. Problem Formulation and Preliminaries

### A. Detection and Attribution of Unseen Builds

An executable is a byte string $x = (b_1, \ldots, b_L)$ with $b_l \in \{0, \ldots, 255\}$, where $L$ denotes the file length in bytes. A virtualization protector is a mapping $O(\cdot)$ that returns an executable $O(x)$ that behaves like $x$ but in which selected code of $x$ is replaced with bytecode for an embedded interpreter. We study two static tasks in which a classifier observes only the bytes of a sample and never executes, unpacks, or disassembles it. In detection, the classifier predicts $y^{\mathrm{d}} \in \{0, 1\}$ (benign or malicious) for an unseen sample. In attribution, the classifier receives $x$ or $O(x)$ without its protection state and predicts its family $y \in \{1, \ldots, K\}$, here the APT group that produced it, where $K$ denotes the number of families.

For attribution, the training data are

$$\mathcal{D} = \{(x_i, O(x_i), y_i)\}_{i \in \mathcal{P}} \cup \{(x_i, y_i)\}_{i \in \mathcal{U}}, \tag{1}$$

where $\mathcal{D}$ denotes the training set, $y_i$ denotes the family of executable $i$, $\mathcal{P}$ denotes the indices of executables that have a protected version, and $\mathcal{U}$ denotes the indices of executables available only in plain form. Executables compiled from one build share their header fields, so a test executable whose build also occurs in training can be recognized rather than attributed. We therefore assign each plain executable $x$ a build key $k(x)$, the tuple of the time stamp, the code size, the size of initialized data, the entry-point address, and the image size in its PE header. We partition the executables into five folds stratified by family, with all executables of one build key in one fold. For fold $q$, the test set contains $x_i$ for every index $i$ in the fold and $O(x_i)$ for every such $i \in \mathcal{P}$. Neither version of a test executable, nor any executable of the same build, enters training or model selection. To test attribution of builds made after the training builds, we also split by time: for each family, the latest builds by compile time stamp that hold about 20% of its executables form the test set. We compare three training protocols on the same folds: Plain (train on $x_i$ only), VM (train on $O(x_i)$ only), and Joint (train on both), and we test every model on plain and on protected test samples.

Our threat model has a defender who trains the classifier and owns a copy of the protector, so that every training executable can be protected, and an adversary who protects a malware sample with the same protector. After protection, the adversary may rewrite every header field that does not affect how Windows loads and runs a user-mode executable, such as the time stamp, the linker version, the section names, and the Rich header (Section III-B). The adversary may also copy these fields from malware of another group, as copied Rich headers have been used to point attribution at another group.[2] Among the loader-relevant fields, the adversary may

[2]Kaspersky GReAT, "The devil's in the Rich header," Securelist, Mar. 8, 2018: https://securelist.com/the-devils-in-the-rich-header/84348/.

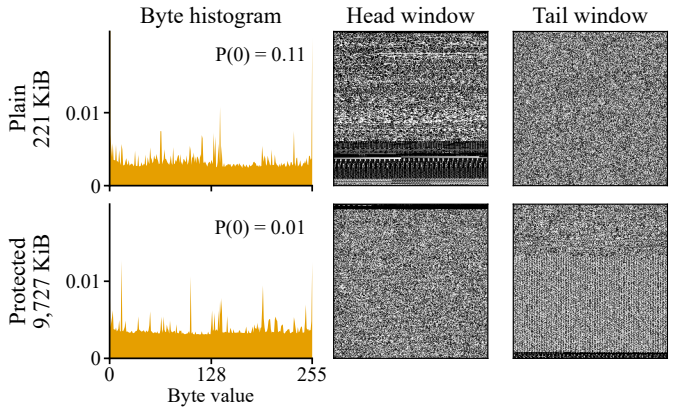



Fig. 2. Byte views of one APT29 executable before and after VMProtect protection. Columns show the byte histogram of the whole file, the head window, and the tail window; $P(0)$ is the fraction of zero bytes.

edit those whose values do not change how the program runs, such as informational flags and the stack and heap sizes. Such functionality-preserving edits of PE files are known from attacks on static malware classifiers [31], [32]. Changes that rebuild the section layout or move the entry point lie outside this model. The adversary knows neither the training data nor the classifier.

For attribution, we report accuracy and the macro-averaged $F_1$ score

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{c=1}^{K} \frac{2P_c R_c}{P_c + R_c}, \tag{2}$$

where $F_1^{\text{macro}}$ denotes the macro-averaged score, and $P_c$ and $R_c$ denote the precision and recall of family $c$, respectively. We compute it on the original test samples and on test samples whose descriptive header fields have been forged or copied from another group or whose loader-relevant fields have been edited (Section IV-D). For detection, we report accuracy, $F_1$, the area under the receiver operating characteristic curve (ROC-AUC), and the true-positive rate (TPR) at a false-positive rate (FPR) of 1%.

Protection changes the bytes that image and byte classifiers read (see Fig. 2): the median size ratio of a protected executable to its source was 57.3, so a protected executable of median size resized to a $128 \times 128$ image averages about 603 bytes per pixel. The PE header, in contrast, keeps much of its content (Section V-C).

### B. Notation, Header Fields, and Compared Methods

EMBER describes each executable by a vector of static features [15], which has 2,381 32-bit floating-point entries in feature version 2 of the EMBER code.[3] The detection images in the archive released with [13] were produced by a renderer that reads the 9,524 bytes of this vector, repeats each byte six times, truncates the sequence at $224 \times 224$ pixels, and rescales the result to $[0, 255]$. Each image therefore contains the first 8,363 bytes of the feature vector. We invert the repetition exactly and fold these bytes, padded with zeros, into a $96 \times 96$ native image for the image networks. We also decode them into the first 2,090 feature values, from the byte histogram to 1,147 of the 1,280 hashed import features. The decoded byte histogram sums to one, as in EMBER, for 93.1% of the training images and 95.3% of the test images. In the other images, the rescaling of the renderer altered the bytes.

TABLE I
NOTATION

| Symbol | Meaning |
|---|---|
| $x$, $L$ | Executable as a byte string; its length in bytes |
| $O(x)$, $F(x)$ | Protected version of $x$; $x$ with forged descriptive fields |
| $y^{\text{d}}$, $y$, $s$ | Detection label; family label; protection state |
| $K$, $c$ | Number of families; family index |
| $\mathcal{P}$, $\mathcal{U}$ | Indices of paired and plain-only executables |
| $k(x)$, $q$ | Build key; fold index |
| $\mathbf{G}$, $M$ | Global statistics view; number of its cells |
| $\mathbf{r}$ | Structure view (97 loader-relevant header fields) |
| $\mathbf{d}$ | Byte-distribution view (514 values) |
| $\mathbf{H}$, $\mathbf{T}$, $W$ | Head window; tail window; window length in bytes |
| $N$, $N_v$, $D$ | Tokens per window; tokens of value view $v$; token dimension |
| $\mathbf{z}$, $\tilde{\mathbf{z}}$ | Projected embeddings of $x$ and $O(x)$ in the paired loss |
| $\tau$, $\alpha$, $\beta$, $\rho$ | Temperature; state weight; pair weight; forging probability |

For attribution, we parse the PE header of every executable from its first 64 KiB. We split its fields into two groups by whether they affect how Windows loads and runs a user-mode executable.[4] The 97 loader-relevant values describe the machine; the number of sections; the PE format; the file characteristics; the sizes of the optional header, of the image, and of the headers; the entry point; the image base; the section and file alignments; the subsystem and its version; the DLL characteristics; the stack and heap sizes; the data directories other than the certificate and debug entries; and the virtual size, virtual address, raw size, raw offset, and characteristics of the first ten sections. The 30 descriptive values, which the loader ignores, are the time stamp; the symbol-table pointer and count; the linker, operating-system, and image versions; the code and data sizes and bases; the certificate and debug directory entries; the names of the first ten sections; and the presence of a Rich header. The Rich header is a block of build-environment metadata between the DOS stub and the PE header [20]. The descriptive values also include the checksum, which Windows validates only for drivers and for DLLs loaded at boot time or into critical processes. The image networks read a global statistics view $\mathbf{G}$, which divides the whole file into $M = 256 \times 256$ cells of consecutive bytes. Its three channels hold, for each cell, the mean byte value, the Shannon entropy of the 256-byte chunk at the cell scaled by 255/8, and the fraction of printable ASCII bytes scaled by 255. The remaining symbols of the paper are defined once in Table I.

We compare the gradient-boosted trees of Section IV-B with eight baseline networks, the multi-view Transformer ObfusFormer of Section IV-C, and a nearest-neighbor rule. We reimplemented the baseline CNN and Mal-CNN of Han *et al.* [13] from the layer table of their paper. The CNN stacks four $3 \times 3$ convolution blocks with batch normalization and max pooling before a 1,024-unit fully connected layer.

[3]Elastic, "EMBER," GitHub repository: https://github.com/elastic/ember.

[4]Microsoft, "PE Format," Microsoft Learn: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format.

Mal-CNN replaces each convolution with a depth-wise separable convolution and adds a $7 \times 7$ spatial attention gate. In attribution, both read the $128 \times 128$ contrast-enhanced images released with that study. Our MalConv [16] reads the concatenated head and tail windows, embeds each byte, applies gated convolutions with kernel size and stride 512, and max-pools over time. The ImageNet-pretrained [33] CNNs ResNet-18 [34], MobileNetV2 [35], and EfficientNet-B0 [36] and Transformers DeiT-Ti [37] (hereafter ViT-Ti) and Swin-T [38] read the global statistics view in attribution. MobileNetV2 and EfficientNet-B0 enter only the detection experiment, and MalConv only the attribution experiment. The nearest-neighbor rule assigns a test sample the family of the training sample with the fewest differing loader-relevant field values. We keep, for each run, the epoch with the highest validation accuracy in detection, or the highest validation macro-$F_1$ (ties broken by validation accuracy) in attribution, and, for the trees, the boosting round with the lowest validation loss.

## IV. Method

Whole-file images average away the fine structure of a protected file, byte sequences mix the regions that protection rewrites with those it keeps, and header fields mix loader-relevant values with values that an adversary can forge. The loader-relevant header fields, in contrast, survive protection, and gradient-boosted trees read them directly. To test whether a learned representation of the raw file can match the trees, we develop the Obfuscation-Paired Multi-View Transformer (ObfusFormer), which reads the loader-relevant header fields, the byte distribution of a whole file, and the two ends of the file through view-specific tokenizers. We train it on each executable together with its protected version and on copies with manipulated header fields. The datasets and splits come first (Section IV-A), followed by the recommended trees (Section IV-B), the Transformer (Section IV-C), and the training and test samples with paired and manipulated headers (Section IV-D).

### A. Detection and Attribution Datasets Grouped by Build

We build one dataset for each task from the archive released with the study of Han *et al.* [13] (Table II). The training split of the detection dataset, which contains EMBER-derived byte images [15], holds 55,000 images, of which 13,755 come from unlabeled EMBER samples that the original scripts stored in the benign folder. We remove these unlabeled images, which leaves 20,524 benign and 20,721 malicious training images. The test split of 2,454 benign and 2,546 malicious images, which the original scripts drew from the EMBER test set, stays unchanged. A fixed random 10% of the training images forms the validation set.

The attribution dataset contains APT malware whose files are named by their SHA-256 hashes. The per-group file counts match the public collection of Boot *et al.* [3] except for one APT1 file.[5] Han *et al.* studied ten of its groups and protected their PE files with VMProtect. We pair each protected file with its plain source through the SHA-256 hash of the source, which gives 2,747 plain and 2,574 protected executables. These executables have only 1,891 distinct build keys. All 395 Equation Group executables share one build key and one file size of 184,320 bytes, so a split grouped by build would place the whole group in one fold. We therefore exclude the Equation Group from the attribution experiments, which leaves 2,352 plain executables of $K = 9$ groups with 1,890 build keys, $|\mathcal{P}| = 2{,}187$, and $|\mathcal{U}| = 165$. We split them into five folds as defined in Section III-A. In each fold, whole builds that hold about 10% of the training executables of each family form the validation set. On the split by time, the latest builds that hold about 10% of the remaining executables of each family form the validation set, and executables whose time stamp lies outside 2000–2020 always stay in training. Its 474 test executables, 404 of them with a protected version, were compiled from 2012 to 2018. No test build of a family is older than its training and validation builds with a valid time stamp.

TABLE II
Datasets. Attribution counts are executables; each protected executable has a plain source, and builds are distinct build keys. *Unlabeled EMBER images stored in the benign folder, removed. †One build, left out

| Detection (EMBER-derived) | | | Attribution (APT) | | | |
|---|---|---|---|---|---|---|
| Split | Benign | Malicious | Family | Plain | Prot. | Builds |
| Train | 20,524 | 20,721 | APT1 | 404 | 399 | 269 |
| Test | 2,454 | 2,546 | APT10 | 228 | 223 | 213 |
| Unlabeled* | 13,755 | – | APT28 | 162 | 160 | 154 |
| | | | APT29 | 263 | 251 | 199 |
| | | | APT30 | 164 | 157 | 145 |
| | | | DarkHotel | 269 | 260 | 218 |
| | | | Energetic Bear | 132 | 127 | 82 |
| | | | Equation Group† | 395 | 387 | 1 |
| | | | Gorgon Group | 345 | 284 | 323 |
| | | | Winnti | 385 | 326 | 287 |
| Total | 22,978 | 23,267 | Used | 2,352 | 2,187 | 1,890 |

### B. Gradient-Boosted Trees on Loader-Relevant Fields

Our attributor reads the loader-relevant header fields of Section III-B with gradient-boosted decision trees, trained with LightGBM [18]. These fields describe how Windows lays out and runs a program—the section table, the data directories, the image base, and the alignments—and VMProtect preserves them, so they carry the family of a protected executable. Forging rewrites only descriptive fields, so it leaves these trees unchanged, and the trees learn no features and train in 3 seconds per fold on the laptop CPU. To separate the family signal that survives protection from the forgeable and byte-level parts, we train four tree models that read, respectively, the loader-relevant fields, all header fields, the descriptive fields, and the byte-distribution view of Section IV-C. In detection, the trees read the 2,090 decoded feature values with the hyperparameters of the EMBER reference model [15].

### C. Multi-View Transformer

ObfusFormer renders each executable into up to four views, tokenizes each view, and fuses the tokens of all views with

[5] cyber-research, "APT Malware Dataset," 2019: https://github.com/cyber-research/APTMalware.

one Transformer encoder (see Fig. 3). The structure view $\mathbf{r} \in \mathbb{R}^{97}$ holds the loader-relevant header fields of Section III-B, with zeros for absent sections. The byte-distribution view $\mathbf{d}$ summarizes the whole file without regard to byte order:

$$\mathbf{d} = [\mathbf{q}^{\mathrm{b}}; \mathbf{q}^{\mathrm{e}}; \log_2 L; \bar{e}_{\mathrm{f}}] \in \mathbb{R}^{514}, \tag{3}$$

where $\mathbf{q}^{\mathrm{b}} \in \mathbb{R}^{256}$ and $\mathbf{q}^{\mathrm{e}} \in \mathbb{R}^{256}$ denote the byte histogram and the byte–entropy histogram of the file, both divided by $L$, and $\bar{e}_{\mathrm{f}}$ denotes the mean Shannon entropy of its 256-byte chunks. The byte–entropy histogram counts the bytes of each chunk in a $16 \times 16$ table of its entropy bin and the high nibble of the byte value, in the manner of Saxe and Berlin [17]. The head window $\mathbf{H}$ holds the first $W = 65{,}536$ bytes (the PE header, the section table, and the start of the sections) and the tail window $\mathbf{T}$ the last $W$ bytes, both zero-padded as a $256 \times 256$ image.

A value tokenizer turns a value view $v \in \{\mathbf{r}, \mathbf{d}\}$ into $N_v$ tokens. Because header fields and histogram entries range over several orders of magnitude, it first compresses each value $a$ to $\mathrm{sign}(a)\log(1+|a|)$ and zero-pads the view to $16N_v$ values. For the structure view, it then encodes each value piecewise linearly over 16 bins whose edges are the quantiles of that field in the training samples [39], hereafter the bin encoding; for the byte-distribution view, it standardizes the values with batch normalization. It splits the encoded values into groups of 16 consecutive values and maps each group with its own linear layer:

$$\mathbf{E}_{v,g} = \mathbf{W}_{v,g}\tilde{\mathbf{v}}_g + \mathbf{b}_{v,g}, \quad g = 1, \ldots, N_v, \tag{4}$$

where $\mathbf{E}_{v,g} \in \mathbb{R}^D$ denotes token $g$ of view $v$, $\tilde{\mathbf{v}}_g$ the encoded values of group $g$, and $\mathbf{W}_{v,g}$, $\mathbf{b}_{v,g}$ its weights and bias; the 97 and 514 values give $N_{\mathbf{r}} = 7$ and $N_{\mathbf{d}} = 33$ tokens.

A square image patch joins bytes one image width apart, whereas executable bytes follow their order. The byte-order tokenizer therefore flattens each window into a sequence of $W$ bytes, embeds each byte value as $\boldsymbol{\phi}_b \in \mathbb{R}^{16}$ to give the input $\mathbf{X}_v \in \mathbb{R}^{16\times W}$ of window $v$, and maps it to $N$ tokens of consecutive bytes:

$$\mathbf{F}_v = \mathcal{S}_R \circ \cdots \circ \mathcal{S}_1\big(\mathrm{GELU}(\mathrm{BN}(\mathrm{Conv}_{r_0}(\mathbf{X}_v)))\big), \tag{5}$$

where $\mathrm{Conv}_{r_0}$ is a one-dimensional convolution with stride $r_0 = 16$ and kernel $2r_0 + 1$, BN is batch normalization, GELU is the Gaussian error linear unit, and each of the $R = 3$ stages $\mathcal{S}_r$ is a depth-wise separable convolution (adapted from MalCNN [13]) with stride 4 and kernel 9, then BN and GELU. With $N = 64$, each token summarizes 1,024 consecutive bytes. An offset attention gate adapts its spatial attention to the offset axis: it multiplies $\mathbf{F}_v$ at each offset by the sigmoid of a kernel-7 convolution over the channel-wise maximum and mean, and a pointwise projection then gives the token sequence $\mathbf{E}_v \in \mathbb{R}^{N\times D}$ of window $v$.

We add to every token a learned positional embedding $\mathbf{P}_v$ of its view and a learned view embedding $\mathbf{u}_v \in \mathbb{R}^D$, and we prepend a class token $\mathbf{t}_{\mathrm{cls}}$:

$$\mathbf{S}^{(0)} = [\mathbf{t}_{\mathrm{cls}}; \mathbf{E}_{v_1} + \mathbf{P}_{v_1} + \mathbf{u}_{v_1}; \ldots; \mathbf{E}_{v_V} + \mathbf{P}_{v_V} + \mathbf{u}_{v_V}], \tag{6}$$

where $\mathbf{S}^{(0)}$ denotes the input sequence of the encoder and $v_1, \ldots, v_V$ denote the $V$ views that a configuration reads. The encoder has the structure of ViT-Ti (DeiT-Ti [37]): 12 pre-norm blocks of three-head self-attention [40] and a multi-layer perceptron, with $D = 192$, initialized from ImageNet-pretrained ViT-Ti while the tokenizers and embeddings train from scratch. The normalized class token after the last block gives the embedding $\mathbf{h} \in \mathbb{R}^D$, which feeds a linear family head $\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{W}_y\mathbf{h} + \mathbf{b}_y)$, a linear protection-state head $\hat{\mathbf{s}}$, and a two-layer projection head $\mathbf{z} = \pi(\mathbf{h})$ used only during training. For detection, ObfusFormer reads the 2,090 decoded feature values as one value view with $N_v = 131$ tokens, and the family head becomes a two-class detection head.

### D. Paired and Manipulated Samples

A protected executable and its plain source carry the same family label but share few byte statistics. We treat each training pair as two versions of one executable and pull their projected embeddings together, in the manner of contrastive learning on augmented views [14] with an InfoNCE loss [41]. Each mini-batch contains $B$ executables drawn without replacement. Every executable $i$ in $\mathcal{P}$ enters the batch with both $x_i$ and $O(x_i)$, and every executable in $\mathcal{U}$ enters with $x_i$ only. The obfuscation-paired loss over the batch is

$$\mathcal{L}_{\mathrm{pair}} = -\frac{1}{2B_p}\sum_{i=1}^{B_p}\Big[\log\frac{\exp(\langle \mathbf{z}_i, \tilde{\mathbf{z}}_i\rangle/\tau)}{\sum_j \exp(\langle \mathbf{z}_i, \tilde{\mathbf{z}}_j\rangle/\tau)} + \log\frac{\exp(\langle \tilde{\mathbf{z}}_i, \mathbf{z}_i\rangle/\tau)}{\sum_j \exp(\langle \tilde{\mathbf{z}}_i, \mathbf{z}_j\rangle/\tau)}\Big], \tag{7}$$

where $\mathcal{L}_{\mathrm{pair}}$ denotes the paired loss, $B_p \le B$ denotes the number of executables of the batch that have a protected version, $\mathbf{z}_i$ and $\tilde{\mathbf{z}}_i$ denote the $\ell_2$-normalized projections of $x_i$ and $O(x_i)$, respectively, $\langle\cdot,\cdot\rangle$ denotes the inner product, $j$ runs over the $B_p$ pairs of the batch, and $\tau$ denotes the temperature. The other pairs of the batch, including pairs of the same family, act as negatives. The full objective is

$$\mathcal{L} = \mathcal{L}^{\epsilon}_{\mathrm{CE}}(\hat{\mathbf{y}}, y) + \alpha\,\mathcal{L}_{\mathrm{CE}}(\hat{\mathbf{s}}, s) + \beta\,\mathcal{L}_{\mathrm{pair}}, \tag{8}$$

where $\mathcal{L}$ denotes the training loss, $y$ and $s$ denote the family and protection state of a sample, $\mathcal{L}^{\epsilon}_{\mathrm{CE}}$ denotes cross-entropy with label smoothing $\epsilon$ [42], $\mathcal{L}_{\mathrm{CE}}$ denotes cross-entropy, both averaged over the samples of the batch, $\alpha$ weights the protection-state loss, and $\beta$ weights the paired loss. Under the Plain and VM protocols, which have no pairs and one protection state, we set $\alpha = \beta = 0$.

The head window holds the descriptive header fields at fixed offsets, so a byte-order tokenizer can learn their values. To keep the configurations of ObfusFormer that read a window from relying on them, we train these configurations also on forged copies. The forging function $F(\cdot)$ draws random values for every descriptive field: the time stamp, the linker, operating-system, and image versions, the checksum, the code and data sizes and bases, and the names of the first ten sections. It zeroes the symbol table, the certificate and debug directory entries, and the Rich header. It leaves every byte of a loader-relevant field unchanged, so $F(x)$ runs like $x$ and has the same structure view. All changed bytes lie in the first 1,088 bytes of a file. We therefore update the other views exactly: the byte histogram, the byte–entropy histogram, and the mean chunk entropy from the changed 256-byte chunks, the head

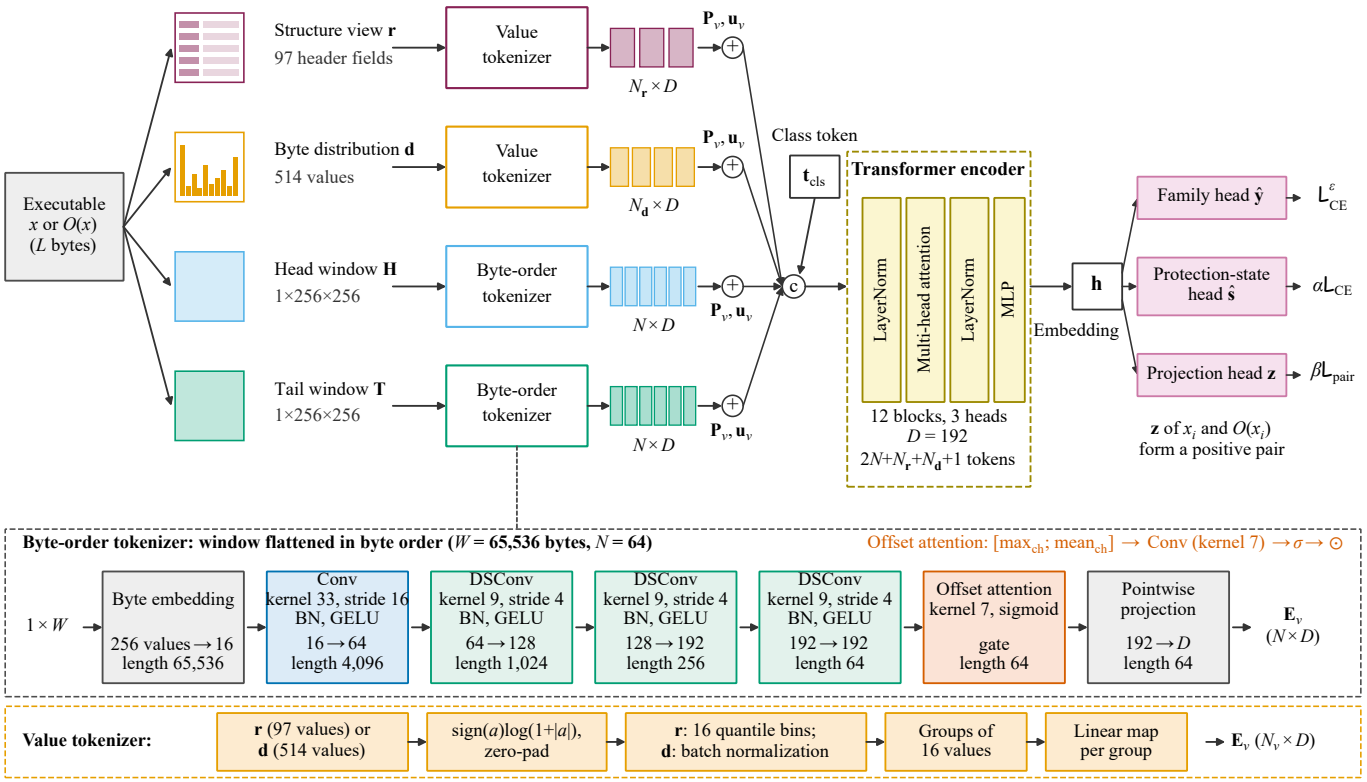



Fig. 3. Architecture of ObfusFormer. Value tokenizers embed the structure view and the byte-distribution view, and each window is flattened in byte order and converted into $N$ tokens by its own byte-order tokenizer. The tokens of all views, with positional and view embeddings, enter a shared Transformer encoder. The class-token embedding **h** feeds the family head, the protection-state head, and the projection head of the obfuscation-paired loss. The zoomed panels show one byte-order tokenizer and the value tokenizer.

window directly, and, for the image networks, the cells of **G** that cover the changed bytes. We precompute eight forged copies of every executable, with different random seeds, and replace each training sample with a randomly chosen copy with probability $\rho$. A ninth copy, with another seed, forms the forged test set of Section III-A. Algorithm 1 summarizes the training procedure.

Two further manipulations change the parsed header fields rather than the bytes, so they apply only to the methods that read header fields. The editing function toggles, each with probability 0.5, the six characteristics flags that the PE specification marks as deprecated or informational for images and the terminal-server-aware DLL flag. It also sets the minor subsystem version to a random value from 0 to 9 when the major version is below 10, draws new stack and heap reserve sizes (64 KiB to 4 MiB) and commit sizes (4 KiB to 64 KiB), and grows the virtual size of the last listed section within its section alignment, so that the image size stays unchanged. As with forging, we precompute nine edited copies of every executable, test on the ninth, and train ObfusFormer on the other eight. The trees train on the original samples together with one edited copy of each. For a false flag, every protected test executable receives all descriptive fields of a random protected training executable of another group.

**Algorithm 1** Training of ObfusFormer with paired and forged samples

**Require:** Training indices $\mathcal{P} \cup \mathcal{U}$, executables $x_i$, protected versions $O(x_i)$, families $y_i$, protection states $s_i$, forged copies $F^{(1)}(\cdot), \ldots, F^{(8)}(\cdot)$; validation set; epochs $E$; probability $\rho$
**Ensure:** Trained parameters $\theta^*$
Initialize the encoder from ViT-Ti and the tokenizers, embeddings, and heads randomly; fit the bin edges of the structure view on the training samples
**for** $t = 1$ **to** $E$ **do**
  **for** each batch $\mathcal{B}$ of $B$ indices from $\mathcal{P} \cup \mathcal{U}$ **do**
    Collect $x_i$ for $i \in \mathcal{B}$ and $O(x_i)$ for $i \in \mathcal{B} \cap \mathcal{P}$
    Replace each sample with a random forged copy with probability $\rho$
    Render the views of the configuration: **r**, **d** with (3), and the windows **H**, **T**
    Tokenize **r** and **d** with (4) and the windows with (5) and the offset attention gate
    Encode the sequence (6) and read **h**, $\hat{\mathbf{y}}$, $\hat{\mathbf{s}}$, **z**
    Compute $\mathcal{L}_{\text{pair}}$ over the pairs in $\mathcal{B}$ with (7)
    Update $\theta$ with AdamW on $\mathcal{L}$ in (8)
  **end for**
  Compute the validation macro-$F_1$ with (2); if it is the best so far, set $\theta^* \leftarrow \theta$
**end for**
**return** $\theta^*$

## V. Experiment

### A. Experimental Setup

We ran the experiments on a laptop with an Intel Core i5-12450H CPU, 16 GB of memory, and an NVIDIA GeForce

TABLE III
TRAINING HYPERPARAMETERS. †VIT-TI, SWIN-T, EFFICIENTNET-B0, AND THE ENCODER OF OBFUSFORMER

| Item | Detection | Attribution |
|---|---|---|
| Epochs | 8 | 15 |
| Batch (samples) | 32 | 16 (Joint: 8 executables) |
| Learning rate | $3 \times 10^{-4}$ | $3 \times 10^{-4}$ |
| CNN, Mal-CNN, MalConv | $5 \times 10^{-4}$ | $5 \times 10^{-4}$ |
| Weight decay, stochastic depth† | 0.05, 0.1 | 0.05, 0.1 |
| Label smoothing $\epsilon$ | 0 | 0.1 |
| ObfusFormer value tokens | 131 | 7 (**r**), 33 (**d**) |
| ObfusFormer window tokens $N$, bins of **r** | – | 64, 16 |
| $\tau$, $\alpha$, $\beta$, $\rho$ | – | 0.1, 0.2, 0.5, 0.5 |
| Swin-T gradient-accumulation steps | 1 | 4 |

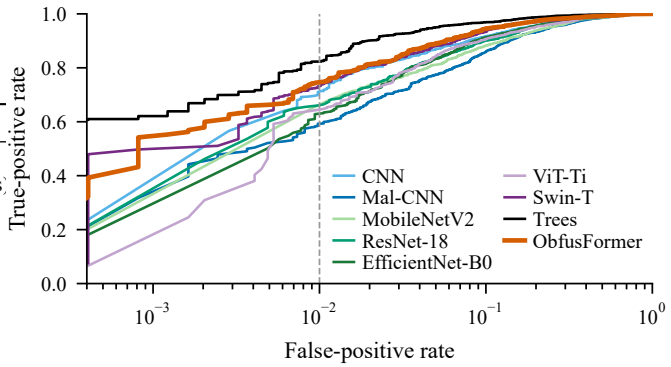



Fig. 4. Receiver operating characteristic curves on the EMBER-derived test images (seed 0). The false-positive rate is on a logarithmic axis; the dashed line marks a false-positive rate of 1%.

TABLE IV
DETECTION ON THE EMBER-DERIVED TEST IMAGES: MEANS OVER THREE SEEDS. THE TREES AND OBFUSFORMER READ THE DECODED FEATURE VALUES, AND THE OTHER NETWORKS READ THE NATIVE IMAGES. THE BEST VALUE IN EACH COLUMN IS IN BOLD

| Method | Acc. (%) | $F_1$ (%) | AUC | TPR@1% (%) |
|---|---|---|---|---|
| *Networks of Han et al. [13]* | | | | |
| CNN | 90.9 | 91.3 | 0.971 | 70.0 |
| Mal-CNN | 88.5 | 88.8 | 0.957 | 62.9 |
| *ImageNet-pretrained networks* | | | | |
| MobileNetV2 | 89.4 | 89.7 | 0.960 | 64.5 |
| ResNet-18 | 89.6 | 90.0 | 0.963 | 63.2 |
| EfficientNet-B0 | 90.1 | 90.4 | 0.964 | 61.8 |
| ViT-Ti | 90.2 | 90.5 | 0.965 | 65.9 |
| Swin-T | 90.7 | 90.9 | 0.970 | 68.7 |
| *Gradient-boosted trees* | | | | |
| Decoded feature values | **94.7** | **94.8** | **0.988** | **82.9** |
| *Transformer (this work)* | | | | |
| ObfusFormer | 91.5 | 91.7 | 0.971 | 72.0 |

RTX 4050 Laptop GPU with 6 GB of memory, under Windows 11, with Python 3.11.15, PyTorch 2.11.0 with CUDA 12.6, timm 1.0.30, scikit-learn 1.8.0, and LightGBM 4.6.0. A second machine had an Intel Core i7-14650HX CPU and an NVIDIA GeForce RTX 4070 Ti SUPER GPU with 16 GB of memory and ran Ubuntu 24.04, Python 3.12.13, PyTorch 2.11.0 with CUDA 12.8, and the same timm version. On it, we ran the attribution runs of the networks and of configurations of ObfusFormer that read more than the structure view, and the detection runs of all networks with seed 2 and of MobileNetV2 and EfficientNet-B0 with seed 1. We ran the detection runs of Mal-CNN and CNN with seed 0 and the Mal-CNN runs on the released augmented images on the CPU in 32-bit precision, and all trees on the CPU of the laptop. We trained all networks with shared hyperparameters and used method-specific values only where marked (Table III). On the GPU, we used bfloat16 autocasting. The optimizer was AdamW [43] with a cosine learning-rate schedule, one warm-up epoch, and gradient-norm clipping at 1.0. The trees in detection used the hyperparameters of the EMBER reference model [15]. In attribution, the trees had 31 leaves, at least 10 samples per leaf, a feature fraction of 0.5, and per-round bagging of 80%, with boosting stopped after 100 of at most 2,000 rounds without an improvement of the validation loss.

We selected the views of ObfusFormer on the validation splits of the Joint protocol (Section V-D). Among the six view configurations, the structure view alone had the highest mean validation macro-$F_1$, 88.3%, against 79.0% for the byte-distribution view and the two windows. In the comparisons, ObfusFormer denotes this configuration, and ObfusFormer-B denotes the configuration with the byte-distribution view and the two windows, which reads the same inputs as the byte-level methods. For detection, batch-normalized value tokens had the highest validation accuracy among five tokenizations with seed 0, 94.3%, against 93.6% for the bin encoding.

### B. Experiment 1: Malware Detection

We trained each detection method with three seeds on the cleaned training images and evaluated it on the 5,000 test images (Table IV). The gradient-boosted trees on the decoded feature values detected malware most accurately, with an accuracy of 94.7% and a ROC-AUC of 0.988. ObfusFormer had an accuracy of 91.5%, 3.2 points below the trees and 0.6 points above CNN, the most accurate image network (90.9%). Its ROC-AUC was 0.971, and its TPR at 1% FPR was 72.0%, against 82.9% for the trees (see Fig. 4). With the unlabeled samples kept in the benign class, as in the original scripts, Mal-CNN had an accuracy of 86.3% with seed 0, against 88.1% on the cleaned images.

### C. Experiment 2: Attribution Under VMProtect

We first measured how the released split of the protected-image dataset affected accuracy. Over three seeds, Mal-CNN had a mean accuracy of 81.2% on the released image-level split, in which 820 of 1,015 test executables also contributed images to training. On a split of the same images grouped by source executable, with the same training settings, the accuracy of our re-implementation fell to 16.5% and its macro-$F_1$ to 18.3%. One executable contributed up to 613 augmented images to the released set.

VMProtect kept much of the PE header of its input. In all 2,574 pairs, the protected file had the time stamp, the linker version, the image base, the section and file alignments, the subsystem, the DLL characteristics, and the code size of its source. VMProtect changed the entry point, the image size, and the number of sections of every file, adding a median of three sections. The section table of 1,966 protected

TABLE V

ATTRIBUTION MACRO-$F_1$ (%) OF APT EXECUTABLES, TRAINING PROTOCOL → TEST VERSION: MEANS ± STANDARD DEVIATIONS OVER FIVE FOLDS (SEED 0), 15 RUNS (SEEDS 0–2, GROUPED FOLDS), OR THREE RUNS (LATER BUILDS). ∗: FORGED DESCRIPTIVE HEADER FIELDS, WHICH CANNOT BE RE-RENDERED IN THE RELEASED IMAGES OF CNN AND MAL-CNN. THE BEST VALUE IN EACH COLUMN IS IN BOLD

| | Folds grouped by build | | | | | | Later builds | |
|---|---|---|---|---|---|---|---|---|
| | Seed 0 | | | | | Seeds 0–2 | Seeds 0–2 | |
| Method | Plain→VM | VM→VM | Joint→Plain | Joint→VM | Joint→VM∗ | Joint→VM | Joint→Plain | Joint→VM |
| *Networks of Han et al. [13]* | | | | | | | | |
| CNN | 5.6±2.1 | 15.2±3.8 | 75.0±3.3 | 11.9±1.1 | – | 12.0±1.1 | 45.3±1.3 | 13.6±0.5 |
| Mal-CNN | 5.0±1.3 | 16.9±4.0 | 73.4±3.2 | 13.2±2.4 | – | 14.5±3.0 | 43.1±1.2 | 12.7±2.8 |
| *Other networks* | | | | | | | | |
| MalConv | 7.8±1.6 | 61.5±5.8 | 82.3±2.3 | 64.1±4.9 | 59.2±5.6 | 64.2±3.9 | 51.3±2.3 | 49.7±1.5 |
| ResNet-18 | 3.9±0.5 | 41.5±1.4 | 83.7±2.2 | 45.9±2.6 | 45.9±2.6 | 46.1±2.3 | 50.9±1.0 | 34.5±0.6 |
| ViT-Ti | 6.0±1.6 | 58.3±2.4 | 81.5±2.4 | 56.9±2.0 | 57.2±1.9 | 57.9±3.5 | 50.9±2.0 | 40.3±3.1 |
| Swin-T | 5.5±1.1 | 62.9±2.5 | 84.1±2.2 | 64.0±3.8 | 64.1±3.8 | 62.7±3.3 | 53.2±4.5 | 45.3±1.6 |
| *Gradient-boosted trees* | | | | | | | | |
| Loader-relevant fields | 16.3±3.4 | 89.5±1.1 | 91.0±1.5 | 90.9±1.1 | **90.9±1.1** | 90.6±1.3 | **59.6±0.3** | **61.0±0.6** |
| All header fields | 43.6±4.2 | **92.4±1.1** | **92.8±1.5** | **92.5±1.5** | 48.3±3.4 | **92.5±1.7** | 50.3±1.2 | 52.2±0.6 |
| Descriptive fields | **65.2±1.8** | 87.9±2.2 | 89.1±3.3 | 88.5±2.6 | 9.0±2.2 | 88.4±2.5 | 27.8±3.0 | 22.5±3.7 |
| Byte distribution | 4.6±1.8 | 74.0±3.8 | 88.3±1.2 | 73.6±2.4 | 68.6±2.8 | 72.9±2.7 | 56.0±0.4 | 46.3±0.3 |
| *Nearest neighbor* | | | | | | | | |
| Loader-relevant fields | – | – | 88.5±0.9 | 85.9±2.8 | 85.9±2.8 | – | 55.2 | 56.0 |
| *Transformer (this work)* | | | | | | | | |
| ObfusFormer | 10.5±7.3 | 86.9±1.5 | 87.6±1.5 | 87.5±0.7 | 87.5±0.7 | 87.3±1.4 | 57.3±0.8 | 57.8±1.8 |
| ObfusFormer-B | 8.2±3.0 | 60.8±3.7 | 80.9±3.8 | 73.7±2.1 | 73.7±2.0 | 70.0±8.1 | 52.9±6.2 | 50.1±4.3 |

files (76.4%) still listed every section name of the source. VMProtect removed the Rich header, which 2,387 of the 2,747 plain executables contained. Builds also repeat: 1,095 of the 2,747 plain executables share their build key with another executable. With folds stratified only by family, 1,020 of the 2,747 plain executables had their build in the training part of another fold. We therefore report all other attribution results on the folds grouped by build and on the split by time.

On unseen builds, the trees on the loader-relevant header fields attributed protected test executables with a macro-$F_1$ of 90.9% under the Joint protocol, and the trees on all header fields with 92.5% (Table V). ObfusFormer, which reads the same loader-relevant fields, had 87.5%, 3.4 points below the trees on these fields; the trees were more accurate in five of the five folds. The nearest-neighbor rule on these fields had 85.9%. The methods that read images or bytes ranked lower: the byte-view configuration ObfusFormer-B had 73.7%, and the baseline networks at most 64.1% (MalConv), down to 11.9% for the CNN of Han *et al.* (see Fig. 5). A bootstrap over the 2,187 protected test executables (10,000 paired, family-stratified resamples with the seed-0 predictions) confirmed both gaps: the trees exceeded ObfusFormer by 3.4 points (95% confidence interval [2.3, 4.5]), and ObfusFormer exceeded the most accurate baseline network by 23.0 points ([20.9, 25.1]). ObfusFormer has 5.7M parameters and trained in 2.0 minutes per fold on the laptop GPU, whereas the trees on the loader-relevant fields trained in 3 seconds per fold on its CPU, so the trees were both the most accurate and the cheapest (see Fig. 6).

The family signal of the loader-relevant fields lay mainly in the layout of a program. The section table and the data directories dominated: permutation importance lowered the protected-test macro-$F_1$ by 49.5 and 28.9 points when they were shuffled, far more than the toolchain fields (the machine,

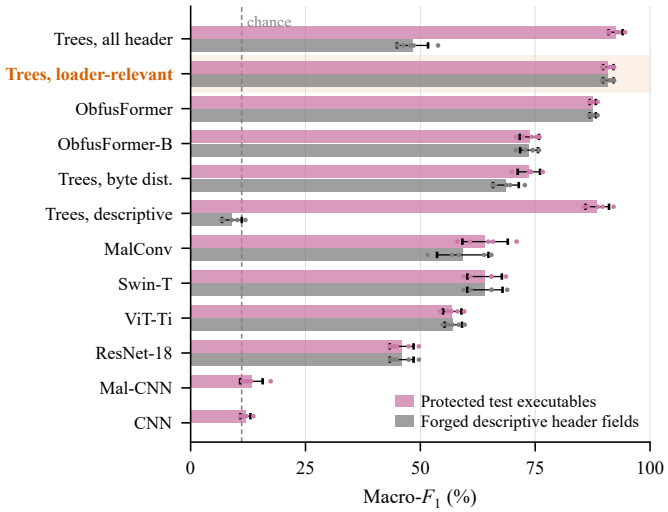



Fig. 5. Attribution under protection. Macro-$F_1$ on the protected test executables under the Joint protocol on the folds grouped by build (seed 0), on the original executables and on executables with forged descriptive header fields. The bars are means over the five folds, the caps the standard deviation, and the dots the folds. The dashed line marks the macro-$F_1$ of a uniformly random classifier of nine families. The CNN and Mal-CNN read released images that cannot be re-rendered and have no forged result; the trees on the loader-relevant fields are highlighted.

the alignments, the image base, and the stack and heap sizes) or the coarse sizes. The trees on the layout fields alone had a protected-test macro-$F_1$ of 90.0%, against 58.3% on the toolchain fields alone, and 80.1% on the coarse file sizes and section count alone, so the surviving signal is largely a size fingerprint of related builds (Fig. S1 of the supplementary material). Trained on plain executables alone, the trees on all loader-relevant fields reached only 16.3% on protected ones, and every network at most 10.5%; restricted to the 49–52 fields that VMProtect kept in at least 95% of the training pairs, they

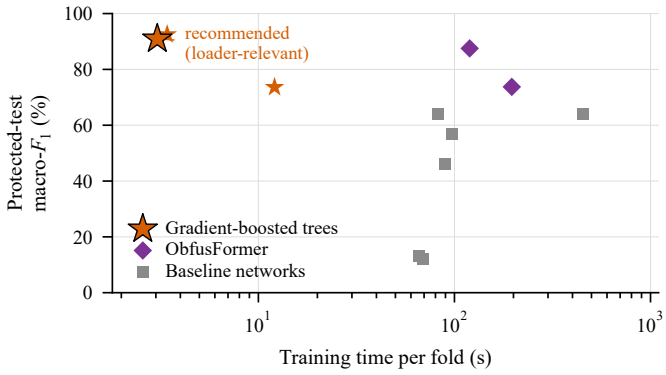



Fig. 6. Protected-test macro-$F_1$ against training time per fold (folds grouped by build, seed 0): the gradient-boosted trees are the most accurate and the cheapest. The trees run on the laptop CPU and the networks and ObfusFormer on its GPU.

TABLE VI
HEADER MANIPULATIONS OF THE PROTECTED TEST EXECUTABLES: MACRO-$F_1$ (%) UNDER THE JOINT PROTOCOL ON FOLDS GROUPED BY BUILD (SEED 0, FIVE FOLDS) AND THE SHARE ATTRIBUTED TO THE GROUP OF THE DONOR OF A FALSE FLAG (%). THE METHODS ON THE LOADER-RELEVANT FIELDS READ NO DESCRIPTIVE FIELD

| Method | Original | Forged | Edited | False flag | Donor |
|---|---|---|---|---|---|
| *Gradient-boosted trees* | | | | | |
| Loader-relevant fields | 90.9 | 90.9 | 85.3 | 90.9 | – |
| Train also on edited copies. | 90.8 | 90.8 | 90.5 | 90.8 | – |
| All header fields | 92.5 | 48.3 | 91.8 | 50.2 | 29.7 |
| Descriptive fields | 88.5 | 9.0 | 88.5 | 0.0 | 100.0 |
| *Nearest neighbor* | | | | | |
| Loader-relevant fields | 85.9 | 85.9 | 84.6 | 85.9 | – |
| *Transformer (this work)* | | | | | |
| ObfusFormer | 87.5 | 87.5 | 68.3 | 87.5 | – |
| Train also on edited copies. | 87.2 | 87.2 | 86.8 | 87.2 | – |

recovered to 89.1%.

Forging the descriptive header fields of the protected test executables lowered the macro-$F_1$ of the trees on all header fields from 92.5% to 48.3% (Table VI). The trees on the descriptive fields alone fell from 88.5% to 9.0%, close to the 11% of a uniformly random classifier of nine families. The forged bytes also lowered the macro-$F_1$ of the methods that read the head window or the byte distribution: MalConv fell from 64.1% to 59.2%, the trees on the byte distribution from 73.6% to 68.6%, and ObfusFormer-B trained without forged copies from 71.2% to 61.0%. Trained with forged copies, ObfusFormer-B had 73.7% on the forged samples. The macro-$F_1$ of the image networks changed by at most 0.3 points, because forging changes only the cells of $\mathbf{G}$ that cover the first 1,088 bytes.

False flags redirected the attribution of the trees that read descriptive fields. With the descriptive fields of a training executable of another group, the trees on all header fields had a macro-$F_1$ of 50.2% and attributed 29.7% of the protected test executables to the group of the donor. The trees on the descriptive fields alone attributed 100.0% of them to that group. Edits of loader-relevant fields lowered the macro-$F_1$ of the methods that read these fields: the trees fell from 90.9% to 85.3%, the nearest-neighbor rule from 85.9% to 84.6%, and ObfusFormer by 19.2 points. Training on edited copies limited the loss to 0.3 points for the trees and 0.4 points for ObfusFormer.

On the split by time, the methods attributed the later builds less accurately than the unseen builds of the grouped folds, except for the CNN of Han *et al.*, which stayed near the 11% of a random classifier on both (Table V). The trees on the loader-relevant fields had a protected-test macro-$F_1$ of 61.0%, 29.6 points below their value on the grouped folds. ObfusFormer had 57.8%, the trees on all header fields 52.2%, and the most accurate baseline network, MalConv, 49.7%. The nearest-neighbor rule had 56.0%, or 65.4% with the validation builds, the latest before the test builds, added to its references.

### D. Ablation of ObfusFormer

We changed one design choice of ObfusFormer at a time, retrained it under the Joint protocol on all five folds, and compared each variant with the configuration it was derived from on the same folds and seeds (Table VII). The structure view alone gave the highest protected-test macro-$F_1$, 87.5% with seed 0; adding the two windows lowered it to 78.7% and the windows and byte distribution to 77.9%, and without the structure view the byte views reached at most 73.7%. Changing only the seed shifted the five-fold mean of ObfusFormer by up to 1.1 points and that of ObfusFormer-B by up to 11.1 points. Over seeds 0–2, the bin encoding and the ImageNet initialization mattered—standardization instead of the bin encoding changed the protected-test macro-$F_1$ of ObfusFormer by −7.3 points and random initialization by −1.7—whereas removing the paired loss or the protection-state head changed it by +0.2 and −0.5 points, less than the seed band (Fig. S2 of the supplementary material). For ObfusFormer-B, the forged copies helped only on forged samples (−9.6 points).

## VI. Discussion

### A. What Survives Virtualization

The experiments yield four findings. First, the measured accuracy of a classifier of protected malware depended on how the data were split. A split by build still rewarded the recognition of related builds: on builds compiled after the training builds, the trees on the loader-relevant fields were 29.6 points less accurate than on the folds grouped by build.

Second, VMProtect protected the code of a program but kept the original section table and data directories in the header, even as it appended sections and moved the entry point. These fields carried 81.9% of the split gain of the trees, and the trees on the toolchain fields alone were less accurate than the trees on the layout fields. These fields possibly reflect how a group builds and links its tools rather than the protected code. The nearest-neighbor rule on the loader-relevant fields came within 1.6 points of ObfusFormer. On later builds, references to the latest earlier builds raised its macro-$F_1$ from 56.0% to 65.4%. Header-based attribution therefore possibly matches a new sample to similar builds of its group rather than learning a rule for the group.

Third, the descriptive header fields gave accurate but forgeable attribution. Forging them or copying them from another

TABLE VII
VIEWS AND TRAINING OF OBFUSFORMER UNDER THE JOINT PROTOCOL ON FOLDS GROUPED BY BUILD: MACRO-$F_1$ (%) ON THE VALIDATION SPLITS AND ON PLAIN, PROTECTED, AND FORGED PROTECTED (∗) TEST EXECUTABLES WITH SEED 0 (MEANS OVER FIVE FOLDS), AND ON PROTECTED TEST EXECUTABLES OVER THE 15 RUNS OF SEEDS 0–2 (MEAN ± STANDARD DEVIATION). CONFIGURATIONS THAT READ A WINDOW TRAIN WITH FORGED COPIES

| | Seed 0 | | | | Seeds 0–2 |
|---|---|---|---|---|---|
| Configuration | Val. | Plain | Prot. | Prot.∗ | Prot. |
| *Views* | | | | | |
| **r** | 88.3 | 87.6 | 87.5 | 87.5 | 87.3±1.4 |
| **d** | 57.0 | 84.5 | 16.9 | 16.8 | 16.9±3.4 |
| **H**, **T** | 72.1 | 72.5 | 67.6 | 67.4 | 68.8±7.2 |
| **d**, **H**, **T** | 79.0 | 80.9 | 73.7 | 73.7 | 70.0±8.1 |
| **r**, **H**, **T** | 83.7 | 82.6 | 78.7 | 78.7 | 76.5±2.8 |
| **r**, **d**, **H**, **T** | 83.1 | 83.9 | 77.9 | 77.9 | 77.4±3.4 |
| *Training* | | | | | |
| ObfusFormer | 88.3 | 87.6 | 87.5 | 87.5 | 87.3±1.4 |
| Remove the paired loss. | 89.0 | 88.5 | 87.9 | 87.9 | 87.5±1.3 |
| Remove the protection-state head. | 87.3 | 86.5 | 86.7 | 86.7 | 86.8±1.8 |
| Replace the bins with standardization. | 81.3 | 81.5 | 80.0 | 80.0 | 80.0±1.6 |
| Initialize the encoder randomly. | 86.7 | 85.8 | 85.3 | 85.3 | 85.6±2.5 |
| ObfusFormer-B | 79.0 | 80.9 | 73.7 | 73.7 | 70.0±8.1 |
| Remove the forged copies. | 78.3 | 80.7 | 71.2 | 61.0 | 72.0±5.5 |
| Remove the paired loss. | 73.0 | 77.5 | 65.0 | 64.9 | 63.3±6.0 |
| Remove the protection-state head. | 73.5 | 75.7 | 64.6 | 64.3 | 68.3±4.8 |

group lowered the macro-$F_1$ of the trees on all header fields by more than 40 points. With the copied fields, the trees attributed 29.7% of the samples to the group of the donor. Networks that read the head window used these fields as well, unless they were trained on forged copies.

Fourth, among the methods on the loader-relevant fields, the trees were the most accurate and ObfusFormer the least robust to edits, which lowered it most unless it was trained on edited copies. On protected test executables, ObfusFormer was 23.4 points above the most accurate baseline network but 3.4 points below the trees, and standardization instead of the bin encoding widened the gap to the trees to 10.9 points. Gorishniy *et al.* [39] reported that piecewise-linear encodings of numerical features improved Transformers and multilayer perceptrons on tabular data. Grinsztajn *et al.* [44] reported that tree ensembles remained more accurate than deep networks, including a Transformer, on medium-sized tabular data. The paired loss and the protection-state head changed ObfusFormer by less than the seed did, possibly because a plain executable and its protected version already share part of their loader-relevant fields.

### B. Remaining Limitations

All protected executables came from one protector, VMProtect, which Han *et al.* [13] applied with options that their paper does not report. The archive of their study contains version 3.5.0. Other virtualizers such as Themida and Code Virtualizer, and protector options that pack the output file, may rebuild the section table and remove the layout that the loader-relevant fields describe. We forged header bytes but copied and edited fields only in the parsed headers, and we did not write or execute any manipulated file. The edits therefore assume that the changed flags and stack and heap sizes leave a program runnable. Larger functionality-preserving changes, such as injected or padded sections [30], [31], would alter the layout fields and were not evaluated. The split by time orders builds by their compile time stamps, which an author can set. The APT labels come from indicators in threat-intelligence reports, and Gray *et al.* [4] noted a lack of assurance about how such labels were assigned. The Equation Group, whose executables form one build, could not be evaluated. The detection images render EMBER feature vectors rather than raw files, so the detection experiment did not test the structure view or the windows. These data also contain no protected benign software, so we could not measure whether protection alone triggers a malicious verdict. Protecting the training executables with several virtualizers and option sets would show whether loader-relevant header fields still identify a family when the protector rebuilds the header. Injecting and padding sections of the test executables would test attribution against layout edits.

## VII. CONCLUSION

We have proposed a build-grouped, forgery-aware evaluation of static attribution of malware protected by virtualization obfuscation, and we have found that gradient-boosted trees on the loader-relevant portable executable header fields attribute it best. On the released protected images, grouping the split by source executable has lowered the accuracy of Mal-CNN from 81.2% to 16.5%. VMProtect has kept the time stamp, linker version, and image base of every executable, and the section table and the data directories have carried most of the trees' importance. On unseen builds, the trees on the loader-relevant fields have attributed protected executables with a macro-$F_1$ of 90.9% in seconds per fold, above the multi-view Transformer we developed (ObfusFormer, 87.5%) and every baseline network. Copying the descriptive fields of another group has lowered the macro-$F_1$ of trees on all header fields to 50.2%, whereas the loader-relevant fields have stayed accurate under forging. On later builds, the most accurate method has reached 61.0%. In detection, the trees on the decoded features have reached an accuracy of 94.7%, 3.2 points above ObfusFormer.

In our future work, we will protect executables with several virtualizers and option sets and date builds by first-seen times to test whether loader-relevant header fields identify the families of new builds.

## REFERENCES

[1] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, vol. 153, p. 102526, 2020.

[2] I. Rosenberg, G. Sicard, and E. David, "DeepAPT: Nation-state APT attribution using end-to-end deep neural networks," in *Artificial Neural Networks and Machine Learning – ICANN 2017*, ser. Lecture Notes in Computer Science, vol. 10614. Cham, Switzerland: Springer, 2017, pp. 91–99.

[3] C. Boot, A. C. Serban, and E. Poll, "Supervised learning for state-sponsored malware attribution," Manuscript, Radboud University, Nijmegen, The Netherlands, 2019, accessed: Oct. 11, 2026. [Online]. Available: https://cs.ru.nl/~aserban/papers/pdf/MalwareStateAttribution.pdf

[4] J. Gray, D. Sgandurra, L. Cavallaro, and J. Blasco Alis, "Identifying authorship in malicious binaries: Features, challenges & datasets," *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–36, 2024.

[5] S. Li, C. Jia, P. Qiu, Q. Chen, J. Ming, and D. Gao, "Chosen-instruction attack against commercial code virtualization obfuscators," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2022.

[6] P. Kochberger, S. Schrittwieser, S. Schweighofer, P. Kieseberg, and E. Weippl, "SoK: Automatic deobfuscation of virtualization-protected applications," in *Proc. 16th Int. Conf. Availability, Reliability and Security (ARES)*, Vienna, Austria, 2021, pp. 1–15.

[7] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proc. 8th Int. Symp. Visualization for Cyber Security (VizSec)*. Pittsburgh, PA, USA: ACM, 2011, pp. 1–7.

[8] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang, and F. Iqbal, "Malware classification with deep convolutional neural networks," in *Proc. 9th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2018, pp. 1–5.

[9] H. Yakura, S. Shinozaki, R. Nishimura, Y. Oyama, and J. Sakuma, "Malware analysis of imaged binary samples by convolutional neural network with attention mechanism," in *Proc. 8th ACM Conf. Data and Application Security and Privacy (CODASPY)*, Tempe, AZ, USA, 2018, pp. 127–134.

[10] S. Seneviratne, R. Shariffdeen, S. Rasnayaka, and N. Kasthuriarachchi, "Self-supervised vision transformers for malware detection," *IEEE Access*, vol. 10, pp. 103 121–103 135, 2022.

[11] I.-W. Jeong, H.-J. Lee, G.-N. Kim, and S.-H. Choi, "MalFormer: A novel vision transformer model for robust malware analysis," *IEEE Access*, vol. 13, pp. 122 671–122 683, 2025.

[12] H. Aghakhani *et al.*, "When malware is packin' heat; limits of machine learning classifiers based on static analysis features," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2020.

[13] S. Han, H. Yun, and Y. Park, "Deep learning for cybersecurity classification: Utilizing depth-wise CNN and attention mechanism on VM-obfuscated data," *Electronics*, vol. 13, no. 17, p. 3393, 2024.

[14] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 1597–1607.

[15] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," arXiv:1804.04637, 2018. [Online]. Available: https://arxiv.org/abs/1804.04637

[16] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, "Malware detection by eating a whole EXE," in *Workshops of the 32nd AAAI Conf. Artificial Intelligence (AI for Cyber Security Workshop)*, New Orleans, LA, USA, 2018, arXiv:1710.09435. [Online]. Available: https://arxiv.org/abs/1710.09435

[17] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," in *Proc. 10th Int. Conf. Malicious Unwanted Softw. (MALWARE)*, 2015, pp. 11–20.

[18] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[19] E. Raff, J. Sylvester, and C. Nicholas, "Learning the PE header, malware detection with minimal domain knowledge," in *Proc. 10th ACM Workshop Artif. Intell. Secur. (AISec)*, Dallas, TX, USA, 2017, pp. 121–132.

[20] G. D. Webster *et al.*, "Finding the needle: A study of the PE32 rich header and respective malware triage," in *Proc. Int. Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. Lecture Notes in Computer Science, vol. 10327, 2017, pp. 119–138.

[21] M. Sharif, A. Lanzi, J. Giffin, and W. Lee, "Automatic reverse engineering of malware emulators," in *Proc. 30th IEEE Symp. Security and Privacy (S&P)*, 2009, pp. 94–109.

[22] D. Xu, J. Ming, Y. Fu, and D. Wu, "VMHunt: A verifiable approach to partially-virtualized binary code simplification," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 442–458.

[23] D. Gibert, N. Totosis, C. Patsakis, Q. Le, and G. Zizzo, "Assessing the impact of packing on static machine learning-based malware detection and classification systems," *Computers & Security*, vol. 156, p. 104495, 2025.

[24] Y. Wu, S. Dou, D. Zou, W. Yang, W. Qiang, and H. Jin, "Contrastive learning for robust Android malware familial classification," *IEEE Transactions on Dependable and Secure Computing*, 2022, early access, Nov. 4, 2022.

[25] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[26] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, 2019, pp. 729–746.

[27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[28] G. Laurenza, R. Lazzeretti, and L. Mazzotti, "Malware triage for early identification of advanced persistent threat activities," *Digital Threats: Research and Practice*, vol. 1, no. 3, pp. 1–17, 2020.

[29] H. Haddadpajouh, A. Azmoodeh, A. Dehghantanha, and R. M. Parizi, "MVFCC: A multi-view fuzzy consensus clustering model for malware threat attribution," *IEEE Access*, vol. 8, pp. 139 188–139 198, 2020.

[30] L. Demetrio, B. Biggio, G. Lagorio, F. Roli, and A. Armando, "Functionality-preserving black-box optimization of adversarial windows malware," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3469–3478, 2021.

[31] L. Demetrio, S. E. Coull, B. Biggio, G. Lagorio, A. Armando, and F. Roli, "Adversarial EXEmples: A survey and experimental evaluation of practical attacks on machine learning for Windows malware detection," *ACM Trans. Privacy Secur.*, vol. 24, no. 4, pp. 1–31, 2021.

[32] H. S. Anderson, A. Kharkar, B. Filar, D. Evans, and P. Roth, "Learning to evade static PE machine learning malware models via reinforcement learning," arXiv:1801.08917, 2018. [Online]. Available: https://arxiv.org/abs/1801.08917

[33] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[35] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[36] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 6105–6114.

[37] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 10 347–10 357.

[38] Z. Liu *et al.*, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 9992–10 002.

[39] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 991–25 004.

[40] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30. Curran Associates, Inc., 2017.

[41] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," arXiv:1807.03748, 2018. [Online]. Available: https://arxiv.org/abs/1807.03748

[42] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[43] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[44] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 507–520.